# 30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

**2026 Edition**

DAY 28

## Performance at Scale — Budgets, CI Regression Gates, Field Monitoring & Incident Response



**Esmaeeil Moradi**
@esmaeeil-moradi700

DAY 28

# Why performance at scale matters in 2026

On your test phone, performance is a bug hunt. On millions of installs across thousands of device models, it's a **system**: what you measure, who owns it, what blocks a merge — and what happens when a release makes things slower.

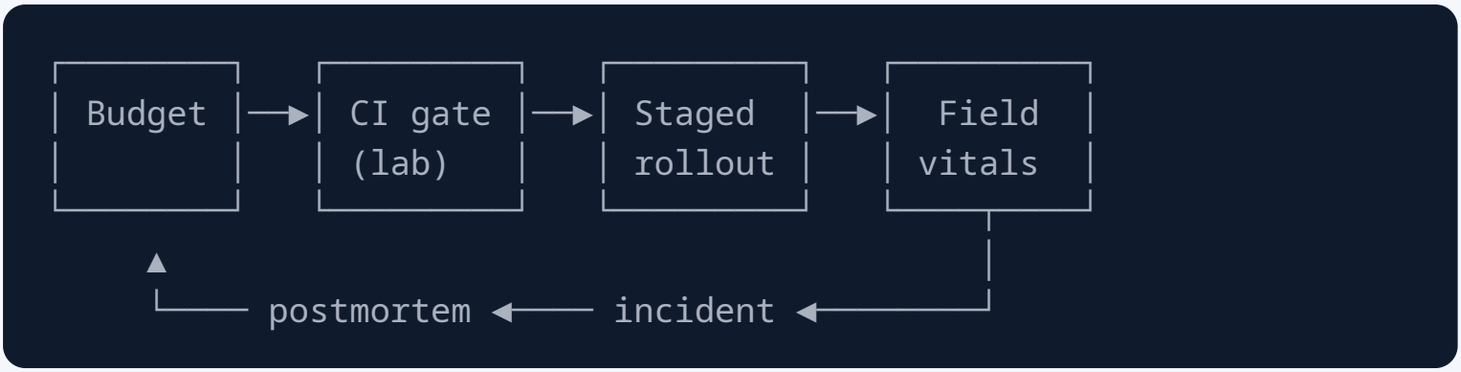
- **The tail is the product:** p90/p99 on low-end devices, not the average on your flagship
- **Lab + field:** Macrobenchmark in CI catches regressions before users do; Android vitals shows what users actually get
- **Budgets with owners** turn "fast" into a number someone is responsible for
- **Google Play enforces it:** apps above the core-vitals bad-behavior thresholds lose visibility
- **Performance problems are production incidents** — they need observability and a playbook

| Level | Questions |
|---|---|
| Junior | Performance at scale · Lab vs field · Metrics that matter |
| Mid | Budgets · CI regression gates · Field traces · Device tiers |
| Senior | Low-end devices · Observability & sampling · Release regressions |
| Architect | Ownership & culture · Incident response · Telemetry pipeline |



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Mental model: a loop, not a sprint




- **Lab** tells you *whether a change* made things slower — controlled, repeatable, before merge
- **Field** tells you *what users get* — real devices, networks and data, after release
- **The loop closes** when every incident ends in a new guardrail: a benchmark, a budget or an alert

**SCALE CHANGES THE QUESTION**

On one phone you ask *"is it fast?"*. At scale you ask *"for whom is it slow, since which version, and which team owns it?"*



**Esmaeeil Moradi**
@esmaeeil-moradi700

DAY 28

# Junior

Performance at scale · Lab vs field · Metrics that matter



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q1. What changes when performance goes "at scale"?

| On your desk | At scale |
|---|---|
| One recent flagship | Thousands of models, many with little RAM |
| Fast office Wi-Fi | Slow mobile data, captive portals, offline |
| Clean test account | Users with years of data and full storage |
| One timing, eyeballed | A distribution: p50, p90, p99 |
| You notice a slowdown | Nobody reports it — users just leave |

A slow path hit by 1% of sessions is invisible on your phone, but with millions of users it is a large group of people every day.

**FOLLOW-UP**

*"Why percentiles, not averages?"* — The mean hides the tail: many fast sessions and a few terrible ones average out to "fine". p90/p99 show how the worst-served users feel.



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q1. Two myths from last year

**INTERVIEW TRAP**

*"Target 60 FPS: every frame in 16 ms."* — Many devices run at 90 or 120 Hz, where the frame deadline is about 11 or 8 ms. Think "no missed deadlines" and measure **jank**, not a fixed FPS number.

**INTERVIEW TRAP**

*"Swap `HashMap` for `SparseArray` to scale."* — Micro-optimizations rarely move field metrics. The big wins are structural: less main-thread work at startup, fewer allocations in scroll paths, smaller payloads. **Measure first**, with the tools from Day 17 and 18.

**WHAT "AT SCALE" REALLY ADDS**

Not new profiling tricks — **process**: budgets, gates, field data sliced by version and device, and clear ownership.



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q2. Lab vs field data

| | Lab (synthetic) | Field (real users) |
|---|---|---|
| Tools | Macrobenchmark, Microbenchmark, Perfetto | Android vitals, Firebase Performance, own telemetry |
| Answers | Did this change make it slower? | What do users actually experience? |
| Strength | Repeatable, before merge, full traces | Real devices, networks, data sizes |
| Weakness | Few devices, scripted journeys | Delayed, noisy, aggregated, sampled |
| Use for | Regression gates in CI | Priorities, release health, alerts |

**INTERVIEW TRAP**

*"Our benchmark says 400 ms, so users see 400 ms."* — Lab numbers compare **builds**; they don't predict user experience. The field distribution is wider and slower, shaped by devices you don't own.



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q2. Testing "at scale" on the client

**INTERVIEW TRAP**

*"Load test the app with thousands of simultaneous users."* — That's a **backend** test. One device runs one user. On the client, scale means **diversity**: large datasets, low-end devices, poor networks, long sessions.

**FOLLOW-UP**

*"How do you test the speed of one function?"* — Jetpack **Microbenchmark** ( `BenchmarkRule` ): it handles warm-up, JIT and clock stability. A `measureTimeMillis` assertion in a JUnit test is mostly noise.

- Lab: benchmark **critical user journeys** — startup, main feed scroll, search, checkout
- Field: the same journeys, measured on real devices and sliced by app version and device tier



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q3. Which metrics does Google Play judge?

Android vitals **core vitals** and their bad-behavior thresholds:

| Core vital | Overall | Per phone model |
|---|---|---|
| User-perceived crash rate | 1.09% | 8% |
| User-perceived ANR rate | 0.47% | 8% |

- Rate = share of daily active users who hit at least one crash / ANR **while actively using** the app
- Above the overall threshold, the app can be shown less on Google Play. Above the per-model threshold, it can be shown less **on that device**, and users may see a warning on the store listing.
- Also watch: slow cold / warm / hot starts (Day 1), slow rendering, frozen frames (over 700 ms), excessive wakeups and wake locks



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q3. Your own metrics — and the trap

Play's vitals are the floor. Add product-level metrics for your **critical user journeys**:

- **TTFD** per key screen (`reportFullyDrawn()` / `ReportDrawnWhen`)
- **Jank** per screen while scrolling
- **Network latency** p90 for the calls that block the UI
- **Download size** and **memory** trends per release

**INTERVIEW TRAP**

*"Crashlytics says 99.5% crash-free users — we're safe."* — Play also counts **ANRs**, uses its own definitions, and judges **each phone model**. An app can be fine overall and above 8% on one popular budget phone.



**Esmaeeil Moradi**
@esmaeeil-moradi700

DAY 28

# Mid-Level

Budgets · CI regression gates · Field traces · Device tiers



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q4. Design a performance budget

A budget = **metric + percentile + device tier + threshold + owner**.

| Budget | Metric | Example rule | Checked by |
|---|---|---|---|
| Cold start | TTID median, mid-tier device | No PR adds more than noise | Macrobenchmark in CI |
| Feed scroll | P90 `frameOverrunMs` | Stays ≤ 0 (deadline met) | Macrobenchmark in CI |
| App size | Download size | Growth needs approval | `bundletool` in CI |
| Network | Bytes per feed load | Tracked per release | Integration test |
| Field start | Cold start p90 per tier | New version ≤ previous | Dashboards, alerts |

Example rules only — derive real numbers from **today's baseline**.

```
bundletool build-apks --bundle=app-release.aab \
  --output=app.apks
bundletool get-size total --apks=app.apks
```



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q4. Budgets that survive contact with reality

- **Absolute budget** (a ceiling users feel) + **relative budget** (no change may regress beyond measured noise)
- **Ratchet:** after an improvement, lower the budget — otherwise the next feature quietly spends the win
- **Per device tier:** low-end can get its own numbers, but always well below Android vitals' "excessive" lines

**INTERVIEW TRAP**

*"Startup budget: 2 s on high-end, 5 s on low-end."* (last year's advice) — 5 s is exactly where Android vitals calls a cold start **excessive**. A budget must sit well below the bad-behavior line, not on it.

**INTERVIEW TRAP**

*"We have budgets."* — Without a named owner and an alert that reaches them, a budget is a wish.



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q5. Catch regressions in CI with Macrobenchmark

```
@RunWith(AndroidJUnit4::class)
class FeedScrollBenchmark {
    @get:Rule val rule = MacrobenchmarkRule()

    @Test fun scrollFeed() = rule.measureRepeated(
        packageName = "com.example.app",
        metrics = listOf(FrameTimingMetric()),
        compilationMode = CompilationMode.Partial(),
        startupMode = StartupMode.WARM,
        iterations = 10,
        setupBlock = { startActivityAndWait() },
    ) {
        val feed = device.findObject(By.res("feed"))
        feed.setGestureMargin(device.displayWidth / 5)
        feed.fling(Direction.DOWN)
        device.waitForIdle()
    }
}
```

- `FrameTimingMetric` reports `frameDurationCpuMs` and `frameOverrunMs` (API 31+) as P50 / P90 / P95 / P99
- `CompilationMode.Partial()` applies your Baseline Profile — the code state real users get



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q5. Make the gate trustworthy

- **Real devices, same model every run.** Emulators don't reflect real rendering or CPU behavior; lock clocks where you can.
- **Release-like target:** non-debuggable, R8 on, `profileable`. The Benchmark library flags debuggable and emulator runs as errors — don't suppress them.
- **Compare distributions:** enough iterations, compared with a rolling baseline from `main`.
- **Detect steps over time**, as Jetpack's own CI does: flag a sustained change across builds instead of failing on one noisy spike.
- Pipeline: results JSON → dashboard → comment on the PR.

**INTERVIEW TRAP**

*"Fail the PR when any metric is 1% slower."* — Run-to-run noise is often bigger than that. Flaky gates get muted, then deleted. Set thresholds from **measured variance** and confirm a step before blocking.



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q6. Measure critical journeys in production

```
class FirebaseTracer(
    private val perf: FirebasePerformance,
) : PerfTracer {
    override suspend fun <T> span(
        name: String,
        attrs: Map<String, String>,
        block: suspend () -> T,
    ): T {
        val trace = perf.newTrace(name)
        attrs.forEach { (k, v) -> trace.putAttribute(k, v) }
        trace.start()
        try { return block() } finally { trace.stop() }
    }
}

// Feature code only sees the PerfTracer interface
val feed = tracer.span(
    name = "feed_load",
    attrs = mapOf("tier" to tier.name),
) { repo.loadFeed() }
```

- A vendor SDK behind **your own interface**: swap Firebase, Sentry or OpenTelemetry without touching features
- Name spans after **user journeys** (`feed_load`, `checkout_submit`), not after classes



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q6. Jank per screen with JankStats

```
// In onCreate(), after setContent { … }
PerformanceMetricsState
    .getHolderForHierarchy(window.decorView)
    .state?.putState("screen", "feed")

jankStats = JankStats.createAndTrack(window) { frame ->
    if (frame.isJank) {
        jankAggregator.record(
            durationNs = frame.frameDurationUiNanos,
            states = frame.states, // e.g. screen=feed
        )
    }
}
// onPause: jankStats.isTrackingEnabled = false
```

**INTERVIEW TRAP**

*"Send an event for every janky frame."* — Thousands of events per session. **Aggregate on device** (counts and histograms per screen) and upload in batches.

**INTERVIEW TRAP**

*"Add the user ID as an attribute to debug single users."* — That's PII, and unbounded cardinality makes dashboards slow and expensive. Use **bounded** dimensions: version, tier, network type, screen.



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q7. Device tiers: classify by hardware

```kotlin
enum class DeviceTier { LOW, MID, HIGH }

// Thresholds are examples: calibrate with your field data
fun Context.deviceTier(): DeviceTier {
    val am = getSystemService(ActivityManager::class.java)
    val mem = ActivityManager.MemoryInfo()
    am.getMemoryInfo(mem)
    val ramGb = mem.totalMem / 1_073_741_824.0
    val mpc = if (Build.VERSION.SDK_INT >= 31) {
        Build.VERSION.MEDIA_PERFORMANCE_CLASS
    } else 0
    return when {
        am.isLowRamDevice || ramGb < 3 -> DeviceTier.LOW
        mpc >= Build.VERSION_CODES.TIRAMISU -> DeviceTier.HIGH
        else -> DeviceTier.MID
    }
}
```

- `isLowRamDevice` is `true` on Android Go–class devices
- **Media performance class** (API 31+) is declared by the manufacturer; `0` means "not declared", not "slow". The Jetpack Core Performance library extends it to older devices.
- Compute once, cache it, and **attach the tier to every metric**



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q7. What to adapt per tier

| Area | Low tier | High tier |
|---|---|---|
| Images | Smaller decode sizes, smaller memory cache | Full resolution |
| Lists | Less prefetch, simpler item layouts | Richer items |
| Motion & effects | Fewer, cheaper animations; no blur | Full effects |
| Video | Lower default bitrate | HD by default |
| Background work | Fewer concurrent jobs | Normal |

**INTERVIEW TRAP**

*"Tier devices by Android version."* — The API level says nothing about RAM or CPU: a new budget phone ships a recent Android on weak hardware. Hard-coded model lists rot too. Use **hardware signals**.

**INTERVIEW TRAP**

*"Low-end mode means fewer features."* — Same features, cheaper rendering. Users on budget phones must finish the same journeys.



**Esmaeeil Moradi**
@esmaeeil-moradi700

DAY 28

# Senior

Low-end devices · Observability & sampling · Release regressions



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q8. Make the app survive on low-end devices

- **Memory is the bottleneck.** A bigger heap means more GC work and more low-memory kills in the background — so more cold starts (Day 1).
- **Decode bitmaps at display size.** Coil and Glide do it when they know the target size; never decode a 12 MP photo for a thumbnail.
- **Baseline Profiles help most on slow CPUs** (Day 1, Day 18).
- **Do less:** lazy init, no extra processes, no background work that doesn't serve the user.

```
class ImageCacheTrimmer(
    private val cache: MemoryCache,
) : ComponentCallbacks2 {
    override fun onTrimMemory(level: Int) {
        // UI_HIDDEN / BACKGROUND: levels still delivered
        val hidden = ComponentCallbacks2.TRIM_MEMORY_UI_HIDDEN
        if (level >= hidden) cache.clear()
    }
    override fun onConfigurationChanged(c: Configuration) = Unit
    override fun onLowMemory() = Unit
}
// Application.onCreate(): registerComponentCallbacks(trimmer)
```



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q8. The low-end traps

**INTERVIEW TRAP**

*"Set `android:largeHeap="true"`."* — It hides the problem on some devices, barely helps on low-RAM ones, and makes your process a bigger target for the low-memory killer. Fix the allocations.

**INTERVIEW TRAP**

*"React to `TRIM_MEMORY_RUNNING_CRITICAL`."* — The `TRIM_MEMORY_RUNNING_*` levels are deprecated since API 34; apps are no longer notified of them. Handle `TRIM_MEMORY_UI_HIDDEN` and `TRIM_MEMORY_BACKGROUND`.

**FOLLOW-UP**

*"How do you cover low-end devices without a drawer full of phones?"* — Keep at least one real low-end device in the benchmark lab, filter field dashboards by tier, and watch `ApplicationExitInfo` with `REASON_LOW_MEMORY` to see how often you're killed. Low-RAM emulators help functional tests, not performance numbers.



# Q9. Observability for a mobile fleet

| Signal | Answers | Mobile example | Cost |
|---|---|---|---|
| Metrics | How much, how often? | Cold start p90 by tier | Low — aggregated |
| Traces (spans) | Where did the time go? | `checkout_submit`: API call, then DB write | Medium — sampled |
| Logs / events | What happened around it? | Cache miss, active flag values | High — needs discipline |
| Crash / ANR / exit reports | Why did it die? | Stack traces, `ApplicationExitInfo` | Low volume, high value |

- **Correlate** everything: the same session id, app version, device tier and flag set on every signal
- **OpenTelemetry** has an Android SDK — a vendor-neutral option for spans and metrics

**INTERVIEW TRAP**

*"Log everything, filter later."* — On mobile, telemetry costs the **user** battery, data and storage, and you pay for ingestion. Every signal needs a question it answers.



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q9. Sampling without lying to yourself

```
data class Sample(val weight: Double, val outlier: Boolean)

class SessionSampler(private val rate: Double) {
    // One decision per session: whole journeys, not pieces
    private val inSample = Random.nextDouble() < rate

    fun decide(durationMs: Long, budgetMs: Long): Sample? =
        when {
            inSample -> Sample(1 / rate, outlier = false)
            // Kept for debugging, excluded from percentiles
            durationMs > budgetMs -> Sample(1.0, outlier = true)
            else -> null
        }
}
```

- **Head sampling per session** keeps journeys whole; **send the weight** so dashboards can re-scale counts
- Always keep **budget-breaking outliers and errors —** in a separate stream, or they bias percentiles
- The sampling rate is a **remote flag**: raise it during an incident, lower it when cost grows



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q10. Did this release make things slower?

- Slice every metric by **app version**; compare the new version with the previous one **over the same days**, on the same device mix
- Watch the staged rollout per version and **halt it** when a core metric crosses its alert line (rollout mechanics: Day 24)
- Tag metrics with **flag and experiment variants** — many regressions ship through remote config, with no new binary
- Reproduce with traces from the field:

**ANDROID 15+ (API 35)**

`ProfilingManager` lets the app request a system trace, heap dump or stack sampling from real users' devices, rate-limited by the system. Newer releases add system-triggered collection, for example around an ANR.



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q10. The cohort trap

**INTERVIEW TRAP**

*"The new version's p90 is worse than last month's, so it regressed."* — Early adopters are a **biased cohort**: devices on Wi-Fi and charging update first, and a fresh update starts before profile-guided compilation has caught up. Compare versions **side by side in the same window**, normalized by device mix.

**INTERVIEW TRAP**

*"Alert when the median startup rises 10% in 24 hours."* (last year's example) — The median hides the tail, and a global number hides a regression on one device family. Alert on **p90 per version and tier**, with enough sessions to be significant.

**FOLLOW-UP**

*"The regression only shows on one OEM's devices. Now what?"* — Check vitals by device model, pull field traces from that cohort, and reproduce on that hardware. Mitigate for that cohort with a flag while you fix.



**Esmaeeil Moradi**
@esmaeeil-moradi700

DAY 28

# Architect

Ownership & culture · Incident response · Telemetry pipeline



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q11. Build a performance culture — who owns what?

| Layer | Mechanism | Owner |
|---|---|---|
| Critical user journeys | 3–5 journeys, each with a budget | Product + engineering leads |
| Budgets & SLOs | Targets per journey and tier | Feature team that owns the journey |
| CI gates | Macrobenchmark, size check, StrictMode in debug | Platform / performance team |
| Field dashboards & alerts | Sliced by version, tier, country | Platform builds, feature teams watch |
| Postmortems | New guardrail after each incident | Team that shipped the change |

A small **platform team** owns tools, CI gates, the telemetry SDK and expertise. **Feature teams own their numbers.**



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q11. Culture, not heroics

- **Performance in design docs:** "what does this cost on a low-end device?" is a standard review question
- **SLO mindset:** when a journey is out of budget, its team fixes it before shipping more features on it
- **Visible dashboards** per team and a regular performance review; celebrate wins with real numbers

**INTERVIEW TRAP**

*"The performance team owns performance."* — Then everyone else ships regressions and waits for them to clean up. The central team provides tools and expertise; ownership stays with the team that ships the code.

**INTERVIEW TRAP**

*"Gate every metric on every PR."* — Alert fatigue. Gate a few stable, high-value metrics; **monitor** the rest.

**FOLLOW-UP**

*"How do you get product to invest in performance?"* — Link budgets to your own business data (conversion, retention, reviews) and to Play's visibility rules for core vitals.



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q12. A release spikes ANRs — walk me through it

| Step | What you do |
|---|---|
| **Detect** | Alerts on crash / ANR / start time per version during rollout — not "a user complained" |
| **Triage** | Scope it: version, device, OS, country, flag variant. New or existing issue? Set severity and an incident lead |
| **Mitigate** | Halt the staged rollout; switch the feature off with a kill switch; revert the server change |
| **Fix** | Hotfix with a **higher versionCode**, through a pre-approved fast path |
| **Learn** | Blameless postmortem; add a guardrail: benchmark, test, alert or lint rule |

**PREPARE BEFORE THE INCIDENT**

A runbook, an on-call rotation, kill switches around risky features, dashboards sliced by version, and a hotfix path you have practiced.



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q12. Incident traps

**INTERVIEW TRAP**

*"Roll back the release in Play Console."* — Halting a rollout stops **new** users from getting it, but users who already updated keep it: Android doesn't downgrade apps. The fix ships as a new release with a higher versionCode — so kill switches must **already be in the binary**.

**INTERVIEW TRAP**

*"Mitigation means fixing the bug."* — First stop the bleeding (halt, flag off), then fix calmly. A rushed hotfix is how incident number two starts.

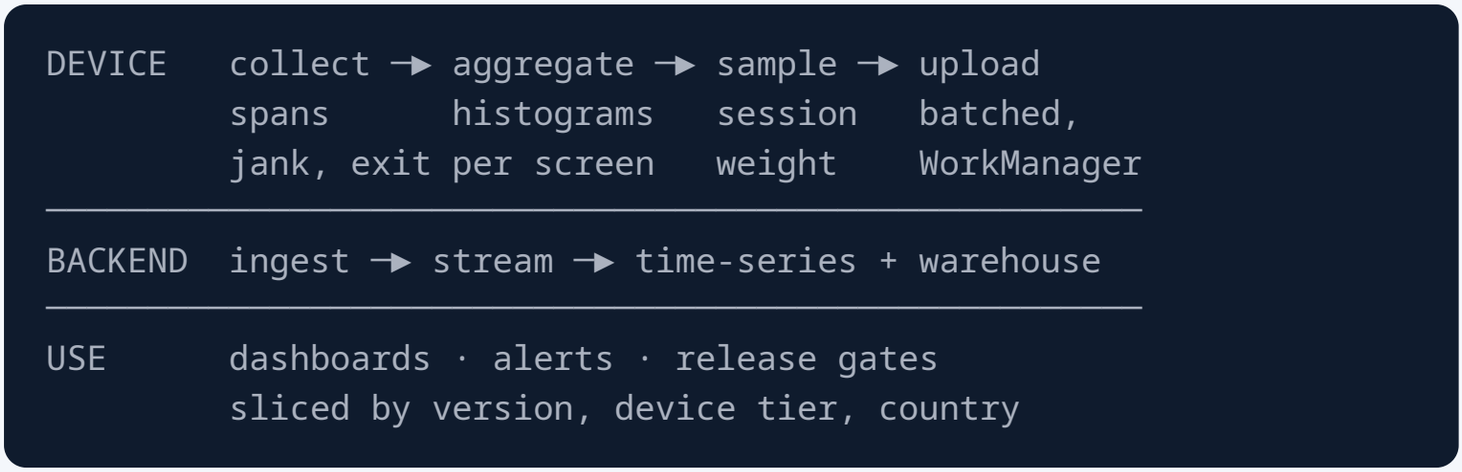
**FOLLOW-UP**

*"What goes into the postmortem?"* — Timeline, impact (users, versions, duration), root cause, why the gates didn't catch it, and owned action items with dates. Blameless: fix the system, not the person.



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q13. Design a telemetry pipeline for millions of devices




- **Aggregate on device:** histograms per metric and screen instead of raw events — less data, battery and privacy risk
- **Upload in batches** with WorkManager (Day 15), with size caps and a drop policy for long offline periods
- **Telemetry must not become the problem:** lazy init off the main thread, bounded memory and disk, a remote kill switch for the SDK itself
- **Privacy by design:** no PII, respect consent (Day 29), declare it in the Data safety form



**Esmaeeil Moradi**
@esmaeeil-moradi700

# Q13. Percentiles don't average

```
class LatencyHistogram(private val boundsMs: LongArray) {
    private val counts = LongArray(boundsMs.size + 1)

    @Synchronized fun record(ms: Long) {
        val i = boundsMs.indexOfFirst { ms <= it }
        counts[if (i == -1) boundsMs.size else i]++
    }

    // Uploaded and merged server-side by summing buckets
    @Synchronized fun snapshotAndReset(): LongArray =
        counts.copyOf().also { counts.fill(0) }
}
```

**INTERVIEW TRAP**

*"Average each device's p90 to get the fleet p90."* — Percentiles don't add up or average. **Merge histograms** (or sketches such as t-digest / DDSketch) and compute percentiles from the merged distribution.

**SENIOR ANSWER IN ONE LINE**

"Performance at scale is a loop: budget it, gate it in CI, watch it in the field by version and device tier, and turn every incident into a new guardrail."



**Esmaeeil Moradi**
@esmaeeil-moradi700

DAY 28 · ARCHITECT

# Day 28 — cheat sheet

- The tail is the product: track **p90/p99 by device tier**, not averages on a flagship.
- Lab (Macrobenchmark) compares builds; field (Android vitals) shows what users get — you need both.
- Core vitals: user-perceived crash rate **1.09%**, ANR rate **0.47%**, **8%** per phone model.
- Budget = metric + percentile + tier + threshold + **owner**. Ratchet it down after every win.
- CI gates: real devices, release-like builds, thresholds from measured variance, step detection.
- Trace journeys behind your own interface; bounded attributes, **no PII**, aggregate on device.
- Tier devices by **hardware signals**, never by API level. Same features, cheaper rendering.
- Sample per session and send weights; keep outliers separately; **merge histograms**, never average percentiles.
- Compare releases side by side in the same window; tag metrics with flag variants.
- Incidents: halt rollout, kill switch, hotfix with a higher versionCode — then a blameless postmortem.

**Day 29 is next → Scalability & global deployment.** Save this post and follow along.



**Esmaeeil Moradi**
@esmaeeil-moradi700